ECO SCUBA — AI Coding Assistant (ACA) Instructions

Complete Project Specification for Lovable.dev

Version 1.0 | Build Target: Production-Ready Full-Stack Web Application

---

COPY/PASTE THIS ENTIRE DOCUMENT AS YOUR LOVABLE.DEV PROMPT. These are complete, self-contained instructions. Build everything described here in a single cohesive project. Do not ask for clarification — make reasonable implementation decisions where details are not specified and document them.

PROJECT OVERVIEW

Build **ECO SCUBA** — a secure, full-stack, AI-powered project proposal writing platform for **Klub vodenih sportova „S.C.U.B.A."** (KVS S.C.U.B.A.), Sarajevo, Bosnia and Herzegovina. The platform orchestrates an intelligent three-protocol AI system (APA + RIP + WA) that guides users through writing professional grant/project proposals in Bosnian, with full collaboration, version control, PDF export, email delivery, and analytics capabilities.

Core Mission: Every feature, every UI decision, every data flow must serve one goal — helping the ECO SCUBA Sekcija write the highest-quality, most professionally formatted, grant-winning project proposals possible.

TECH STACK

Frontend

- **Framework:** React 18 + TypeScript
- **Build Tool:** Vite
- **Styling:** Tailwind CSS v3 + custom CSS variables (no component libraries that impose visual identity — build custom components)
- **UI Components:** Radix UI primitives (headless) + custom styling
- **Rich Text / HTML Preview:** react-contenteditable or @uiw/react-md-editor for section editing
- **PDF Generation:** @react-pdf/renderer (primary) + html2pdf.js (fallback for pixel-perfect form replication)
- **Charts/Analytics:** Recharts
- **Animations:** Framer Motion
- **Forms:** React Hook Form + Zod validation
- **File Upload:** react-dropzone

- **Email:** Resend API (via Supabase Edge Function)
- **Icons:** Lucide React

Backend / BaaS

- **Platform:** Supabase
    - PostgreSQL database
    - Row Level Security (RLS) on all tables
    - Supabase Auth (email/password + Google OAuth)
    - Supabase Storage (PDF uploads, form uploads, generated PDFs)
    - Supabase Edge Functions (Deno) for AI API calls, PDF generation, email sending
    - Supabase Realtime (for collaboration features)

AI Integration

- **Provider:** Anthropic Claude API (claude-sonnet-4-6-20251001)
- **Pattern:** Streaming responses via Supabase Edge Functions (never expose API key to frontend)
- **Context Management:** Full conversation history stored in database; passed on each API call

Authentication

- Supabase Auth
- Google OAuth 2.0
- Email/password with email verification
- JWT session management (Supabase handles)

DESIGN SYSTEM & UI/UX DIRECTION

Aesthetic Direction: "Deep Water Professionalism"

The visual identity draws from the underwater world — deep ocean blues, bioluminescent accents, the precision of scientific instruments, and the calm authority of professional documentation. This is NOT a playful diving app — it is a serious professional tool that happens to belong to a diving club.

Color Palette (CSS Variables)

```
:root {
  /* Backgrounds */
  --bg-primary: #0a0f1e;       /* Deep ocean — main background */
```

```
--bg-secondary: #0d1829;     /* Slightly lighter — cards, panels */
--bg-tertiary: #111f35;      /* Hover states, subtle contrast */
--bg-surface: #162340;       /* Elevated surfaces, modals */
--bg-glass: rgba(13, 24, 41, 0.85); /* Glassmorphism */

/* Brand Colors */
--brand-primary: #0ea5e9;    /* Electric ocean blue — primary actions */
--brand-secondary: #06b6d4;  /* Cyan — secondary actions */
--brand-accent: #38bdf8;     /* Light blue — highlights */
--brand-deep: #0369a1;       /* Deep blue — hover states */

/* Bioluminescent Accents */
--accent-glow: #00d4ff;      /* Glowing teal — active states, indicators */
--accent-success: #10b981;   /* Emerald — success states */
--accent-warning: #f59e0b;   /* Amber — warnings, pending states */
--accent-danger: #ef4444;    /* Red — errors, critical actions */

/* Text */
--text-primary: #f0f6ff;     /* Near white — primary text */
--text-secondary: #94b4d4;   /* Muted blue-white — secondary text */
--text-muted: #4d7094;       /* Dark muted — disabled, placeholders */
--text-inverse: #0a0f1e;     /* Dark — text on light backgrounds */

/* Borders */
--border-subtle: rgba(14, 165, 233, 0.12);
--border-default: rgba(14, 165, 233, 0.25);
--border-strong: rgba(14, 165, 233, 0.5);

/* Shadows & Glows */
--shadow-sm: 0 2px 8px rgba(0, 0, 0, 0.3);
--shadow-md: 0 4px 20px rgba(0, 0, 0, 0.4);
```

```
  --shadow-glow: 0 0 20px rgba(14, 165, 233, 0.15);
  --shadow-glow-strong: 0 0 40px rgba(14, 165, 233, 0.25);
}
```

Typography

```
/* Import from Google Fonts */
@import
url('https://fonts.googleapis.com/css2?family=Syne:wght@400;600;700;800&family=DM+Sans:ital,opsz,wght@0,9..40,300;0,9..40,400;0,9..40,500;0,9..40,600;1,9..40,300&display=swap');

:root {
  --font-display: 'Syne', sans-serif;    /* Headings, navigation, labels */
  --font-body: 'DM Sans', sans-serif;    /* Body text, paragraphs, UI text */
  --font-mono: 'JetBrains Mono', monospace; /* Code, IDs, technical data */
}
```

Design Principles

1. **Depth over flatness** — Cards have subtle shadows and glassmorphism; backgrounds have gradient depth
2. **Information hierarchy** — Critical information (project name, status, next action) always visually dominant
3. **Precision** — Every pixel intentional; consistent 4px grid; precise spacing
4. **Motion with purpose** — Animations communicate state (loading, success, transitions) — never decorative noise
5. **Accessible contrast** — WCAG AA minimum on all text/background combinations despite dark theme

Component Design Patterns

- **Cards:** bg-surface, border border-subtle, rounded-xl, subtle inner glow on hover (box-shadow: var(--shadow-glow))
- **Buttons (Primary):** bg-brand-primary, text-inverse, rounded-lg, Framer Motion whileHover scale 1.02, whileTap scale 0.98
- **Buttons (Secondary):** border border-brand-primary, text-brand-primary, transparent background
- **Buttons (Ghost):** Transparent, text-text-secondary, hover bg-bg-tertiary
- **Input fields:** bg-bg-secondary, border border-subtle, focus border-brand-primary + glow ring

- **Status badges:** Rounded pill, color-coded by status
- **Section cards (APA sections):** Left border accent in brand color, expandable, approval state visually clear
- **Loading states:** Skeleton screens with shimmer animation (never spinners for content loading)
- **Empty states:** Illustrative, encouraging, action-oriented

---

DATABASE SCHEMA (Supabase PostgreSQL)

```
-- ============================================
-- USERS & AUTHENTICATION
-- ============================================
-- auth.users is managed by Supabase Auth

CREATE TABLE public.profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  email TEXT NOT NULL,
  full_name TEXT,
  avatar_url TEXT,
  organization TEXT DEFAULT 'KVS S.C.U.B.A.',
  role TEXT DEFAULT 'member' CHECK (role IN ('admin', 'member', 'guest')),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- ============================================
-- PROJECTS
-- ============================================
CREATE TABLE public.projects (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title TEXT NOT NULL,
  slug TEXT UNIQUE,
  owner_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
```

```
status TEXT DEFAULT 'draft' CHECK (status IN (
  'draft', 'in_progress', 'review', 'completed', 'archived'
)),

-- Project metadata (from APA data collection)
donor_name TEXT,
donor_deadline TIMESTAMPTZ,
priority_area TEXT,
project_locations JSONB DEFAULT '[]',
project_duration_months INTEGER,
project_start_date DATE,
project_end_date DATE,
total_budget_km NUMERIC(12,2),
requested_budget_km NUMERIC(12,2),
own_contribution_km NUMERIC(12,2),
direct_beneficiaries INTEGER,

-- Form template
form_template_url TEXT,  -- Supabase Storage URL of uploaded PDF form
form_template_analysis JSONB,  -- Pixel-perfect format analysis stored as JSON

-- APA State Register (full JSON state)
apa_state JSONB DEFAULT '{}',

-- RIP Research Package (full research data)
rip_data JSONB DEFAULT '{}',

-- Final assembled document
final_html TEXT,
final_pdf_url TEXT,  -- Supabase Storage URL
```

```
  -- Metadata
  language TEXT DEFAULT 'bs' CHECK (language IN ('bs', 'en')),
  tags TEXT[] DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- ============================================
-- PROJECT SECTIONS
-- ============================================
CREATE TABLE public.project_sections (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID NOT NULL REFERENCES public.projects(id) ON DELETE CASCADE,
  section_number TEXT NOT NULL,  -- e.g., "0", "1", "2", "2.1", "3", ...
  section_key TEXT NOT NULL,     -- e.g., "naslovna_strana", "uvod", "sazetak"
  section_title_bs TEXT NOT NULL, -- Always Bosnian title
  content_html TEXT,             -- WA-generated HTML content
  status TEXT DEFAULT 'pending' CHECK (status IN (
    'pending', 'generating', 'awaiting_approval', 'approved', 'revision_requested'
  )),
  version INTEGER DEFAULT 1,
  is_optional BOOLEAN DEFAULT FALSE,
  assigned_to UUID REFERENCES public.profiles(id),  -- For collaboration

  -- Approval tracking
  approved_by UUID REFERENCES public.profiles(id),
  approved_at TIMESTAMPTZ,

  -- Sort order
  display_order INTEGER NOT NULL,
```

```
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),

  UNIQUE(project_id, section_key)
);

-- ============================================
-- SECTION REVISIONS (Full version history)
-- ============================================
CREATE TABLE public.section_revisions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  section_id UUID NOT NULL REFERENCES public.project_sections(id) ON DELETE
CASCADE,
  project_id UUID NOT NULL REFERENCES public.projects(id) ON DELETE CASCADE,
  version INTEGER NOT NULL,
  content_html TEXT NOT NULL,
  change_description TEXT,
  change_requested_by UUID REFERENCES public.profiles(id),
  generated_by TEXT DEFAULT 'wa_protocol',  -- 'wa_protocol' or 'user'
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- ============================================
-- CHANGE LOG (APA Change Propagation Log)
-- ============================================
CREATE TABLE public.change_log (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID NOT NULL REFERENCES public.projects(id) ON DELETE CASCADE,
  requested_by UUID NOT NULL REFERENCES public.profiles(id),
  change_description TEXT NOT NULL,
  affected_sections TEXT[] DEFAULT '{}',  -- section_keys affected
```

```
  apa_analysis TEXT,   -- APA's analysis of the change
  apa_elaboration TEXT, -- APA's elaboration and implementation plan
  status TEXT DEFAULT 'pending' CHECK (status IN (
    'pending', 'approved_by_user', 'applied', 'rejected'
  )),
  applied_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- ============================================
-- AI CONVERSATIONS
-- ============================================
CREATE TABLE public.ai_conversations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID NOT NULL REFERENCES public.projects(id) ON DELETE CASCADE,
  section_id UUID REFERENCES public.project_sections(id),
  protocol TEXT NOT NULL CHECK (protocol IN ('APA', 'RIP', 'WA', 'SYSTEM')),
  messages JSONB DEFAULT '[]',  -- Full message history for Claude API
  token_count INTEGER DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- ============================================
-- PROJECT COLLABORATORS
-- ============================================
CREATE TABLE public.project_collaborators (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID NOT NULL REFERENCES public.projects(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
```

```
  role TEXT NOT NULL DEFAULT 'editor' CHECK (role IN ('owner', 'editor', 'reviewer',
'viewer')),
  invited_by UUID REFERENCES public.profiles(id),
  invited_at TIMESTAMPTZ DEFAULT NOW(),
  accepted_at TIMESTAMPTZ,
  status TEXT DEFAULT 'pending' CHECK (status IN ('pending', 'accepted', 'declined')),
  section_assignments TEXT[] DEFAULT '{}',  -- section_keys this collaborator is
responsible for

  UNIQUE(project_id, user_id)
);

-- ============================================
-- COLLABORATION TASKS
-- ============================================
CREATE TABLE public.collaboration_tasks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID NOT NULL REFERENCES public.projects(id) ON DELETE CASCADE,
  section_id UUID REFERENCES public.project_sections(id),
  assigned_to UUID NOT NULL REFERENCES public.profiles(id),
  assigned_by UUID NOT NULL REFERENCES public.profiles(id),
  task_type TEXT NOT NULL CHECK (task_type IN (
    'review_section', 'write_section', 'provide_data', 'approve_section', 'finalize_budget'
  )),
  title TEXT NOT NULL,
  description TEXT,
  due_date TIMESTAMPTZ,
  status TEXT DEFAULT 'open' CHECK (status IN ('open', 'in_progress', 'done',
'overdue')),
  priority TEXT DEFAULT 'normal' CHECK (priority IN ('low', 'normal', 'high', 'urgent')),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
```

```
);

-- ============================================
-- NOTIFICATIONS
-- ============================================
CREATE TABLE public.notifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  project_id UUID REFERENCES public.projects(id),
  type TEXT NOT NULL CHECK (type IN (
    'section_awaiting_approval', 'collaborator_invited', 'task_assigned',
    'task_overdue', 'project_completed', 'change_applied', 'mention'
  )),
  title TEXT NOT NULL,
  message TEXT,
  is_read BOOLEAN DEFAULT FALSE,
  action_url TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- ============================================
-- PROJECT TEMPLATES (Historical projects as learning data)
-- ============================================
CREATE TABLE public.project_templates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title TEXT NOT NULL,
  description TEXT,
  source_project_id UUID REFERENCES public.projects(id),
  category TEXT,  -- 'ecology', 'education', 'sport', 'community'
  donor_type TEXT,  -- 'embassy', 'eu', 'usaid', 'municipal', 'private'
  sections_data JSONB DEFAULT '{}',  -- Anonymized section content for learning
```

```
  quality_score NUMERIC(3,1),  -- 1.0-10.0 rating
  is_public BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- ============================================
-- ROW LEVEL SECURITY
-- ============================================
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.projects ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.project_sections ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.section_revisions ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.change_log ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.ai_conversations ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.project_collaborators ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.collaboration_tasks ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.notifications ENABLE ROW LEVEL SECURITY;

-- Profiles: users can read/update their own profile
CREATE POLICY "Users can view own profile" ON public.profiles FOR SELECT USING
(auth.uid() = id);
CREATE POLICY "Users can update own profile" ON public.profiles FOR UPDATE USING
(auth.uid() = id);

-- Projects: owners and collaborators can access
CREATE POLICY "Project access" ON public.projects FOR ALL USING (
  auth.uid() = owner_id OR
  auth.uid() IN (
    SELECT user_id FROM public.project_collaborators
    WHERE project_id = id AND status = 'accepted'
  )
);
```

```
-- Sections: same as project access
CREATE POLICY "Section access via project" ON public.project_sections FOR ALL
USING (
  project_id IN (
    SELECT id FROM public.projects WHERE
    auth.uid() = owner_id OR
    auth.uid() IN (
      SELECT user_id FROM public.project_collaborators
      WHERE project_id = projects.id AND status = 'accepted'
    )
  )
);

-- Apply similar policies to all other tables referencing project_id

-- ============================================
-- INDEXES
-- ============================================
CREATE INDEX idx_projects_owner ON public.projects(owner_id);
CREATE INDEX idx_projects_status ON public.projects(status);
CREATE INDEX idx_sections_project ON public.project_sections(project_id);
CREATE INDEX idx_sections_status ON public.project_sections(status);
CREATE INDEX idx_collaborators_user ON public.project_collaborators(user_id);
CREATE INDEX idx_notifications_user_unread ON public.notifications(user_id, is_read);
CREATE INDEX idx_tasks_assigned ON public.collaboration_tasks(assigned_to, status);

-- ============================================
-- FUNCTIONS & TRIGGERS
-- ============================================
-- Auto-update updated_at
```

```
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN NEW.updated_at = NOW(); RETURN NEW; END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER update_projects_updated_at BEFORE UPDATE ON public.projects
  FOR EACH ROW EXECUTE FUNCTION update_updated_at();
CREATE TRIGGER update_sections_updated_at BEFORE UPDATE ON
public.project_sections
  FOR EACH ROW EXECUTE FUNCTION update_updated_at();

-- Auto-create profile on user signup
CREATE OR REPLACE FUNCTION handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO public.profiles (id, email, full_name, avatar_url)
  VALUES (NEW.id, NEW.email, NEW.raw_user_meta_data->>'full_name',
NEW.raw_user_meta_data->>'avatar_url');
  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER on_auth_user_created AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE FUNCTION handle_new_user();
```

APPLICATION ARCHITECTURE

Directory Structure

```
src/
├── components/
│   ├── auth/
│   │   ├── LoginForm.tsx
│   │   ├── RegisterForm.tsx
```

```
│   │   └── AuthGuard.tsx
│   ├── layout/
│   │   ├── AppShell.tsx
│   │   ├── Sidebar.tsx
│   │   ├── TopNav.tsx
│   │   └── NotificationBell.tsx
│   ├── projects/
│   │   ├── ProjectCard.tsx
│   │   ├── ProjectList.tsx
│   │   ├── NewProjectWizard.tsx
│   │   └── ProjectStatusBadge.tsx
│   ├── editor/
│   │   ├── ProjectEditor.tsx          -- Main editor layout
│   │   ├── SectionCard.tsx            -- Individual section with APA/WA
│   │   ├── SectionApprovalBar.tsx     -- Approve/Edit/Rewrite controls
│   │   ├── AIStreamingOutput.tsx      -- Real-time AI text streaming
│   │   ├── HTMLPreview.tsx            -- Rendered HTML section preview
│   │   ├── ChangeRequestPanel.tsx     -- FIX-06 change protocol UI
│   │   ├── APADataCollection.tsx      -- Initial data collection wizard
│   │   └── DisclaimerBanner.tsx       -- FIX-05 responsibility clause
│   ├── collaboration/
│   │   ├── CollaboratorList.tsx
│   │   ├── InviteCollaborator.tsx
│   │   ├── TaskBoard.tsx
│   │   └── TaskCard.tsx
│   ├── pdf/
│   │   ├── PDFExporter.tsx
│   │   ├── PDFPreview.tsx
│   │   └── FormUploader.tsx
│   ├── analytics/
│   │   ├── Dashboard.tsx
```

```
│   │   ├── ProjectProgressChart.tsx
│   │   ├── SectionStatusPie.tsx
│   │   ├── TasksOverview.tsx
│   │   └── ActivityTimeline.tsx
│   └── ui/                    -- Reusable design system components
│       ├── Button.tsx
│       ├── Card.tsx
│       ├── Badge.tsx
│       ├── Input.tsx
│       ├── Modal.tsx
│       ├── Tooltip.tsx
│       ├── SkeletonLoader.tsx
│       └── GlowEffect.tsx
├── pages/
│   ├── auth/
│   │   ├── Login.tsx
│   │   └── Register.tsx
│   ├── Dashboard.tsx
│   ├── ProjectsList.tsx
│   ├── ProjectEditor.tsx
│   ├── Analytics.tsx
│   ├── Collaboration.tsx
│   └── Settings.tsx
├── hooks/
│   ├── useAuth.ts
│   ├── useProject.ts
│   ├── useSection.ts
│   ├── useAIStream.ts
│   ├── useCollaboration.ts
│   └── useNotifications.ts
├── lib/
```

```
│   ├── supabase.ts
│   ├── ai-client.ts              -- Claude API integration
│   ├── pdf-generator.ts          -- PDF generation logic
│   ├── email-sender.ts           -- Email integration
│   └── apa-system-instructions.ts -- Full APA+RIP+WA system prompt
├── store/
│   ├── projectStore.ts           -- Zustand store
│   ├── uiStore.ts
│   └── notificationStore.ts
└── types/
    └── index.ts                  -- All TypeScript interfaces
```

SUPABASE EDGE FUNCTIONS

Function 1: ai-generate-section

// supabase/functions/ai-generate-section/index.ts

// Handles all Claude API calls with streaming

// Receives: { project_id, section_key, protocol, conversation_history, context }

// Returns: Server-Sent Events stream of Claude response

// Security: Validates auth token, checks project access via RLS

Function 2: generate-pdf

// supabase/functions/generate-pdf/index.ts

// Receives: { project_id, sections_html[], format_config }

// Uses puppeteer-core or html-to-pdf for server-side PDF generation

// Stores result in Supabase Storage

// Returns: { pdf_url, file_size }

Function 3: send-project-email

// supabase/functions/send-project-email/index.ts

// Uses Resend API

// Receives: { project_id, recipient_email, recipient_name, message, attach_pdf }

// Sends professional HTML email with PDF attachment

// Returns: { success, message_id }

Function 4: process-form-upload

// supabase/functions/process-form-upload/index.ts

// Receives uploaded PDF form

// Uses Claude Vision to analyze form structure

// Returns: { form_sections, format_analysis, section_labels }

CORE FEATURES — DETAILED SPECIFICATION

FEATURE 1: AUTHENTICATION

Login Page (/login)

- **Full-screen immersive design** — deep ocean background with subtle animated particle system (CSS only, no canvas)
- ECO SCUBA logo centered, large
- Email/password form with show/hide password toggle
- **"Prijava putem Google računa"** button (Google OAuth via Supabase)
- **"Nemate račun? Registrujte se"** link
- Forgot password flow
- Loading state on submit with disabled form
- Error handling: wrong password, unverified email, account not found

Register Page (/register)

- Same visual design as login
- Fields: Full name, email, password, confirm password
- Email verification flow
- Redirect to dashboard after verification

Auth Guard

- All routes except /login and /register require authentication
- Unauthenticated users redirected to /login with return URL preserved
- Session persistence via Supabase persistSession: true

Google OAuth

// Implementation

const { error } = await supabase.auth.signInWithOAuth({

```
  provider: 'google',
  options: {
    redirectTo: `${window.location.origin}/auth/callback`,
    queryParams: { access_type: 'offline', prompt: 'consent' }
  }
});
```

FEATURE 2: DASHBOARD (/dashboard)

Layout: App Shell

- **Left sidebar** (collapsible, 260px expanded / 64px collapsed):
  - ECO SCUBA logo + wordmark at top
  - Navigation items with icons: Dashboard, Projekti, Analitika, Saradnja, Podešavanja
  - User avatar + name at bottom with logout option
  - Active state: left accent border in --brand-primary, background --bg-tertiary
  - Animated collapse/expand with Framer Motion
- **Top navigation bar** (64px height):
  - Page title (dynamic)
  - Search bar (global search across projects and sections)
  - Notification bell with unread count badge
  - Quick action button: "Novi projekat"

Dashboard Content

- **Hero row (3 stat cards):**
  - Total active projects (with trend vs. last month)
  - Sections awaiting approval (clickable — goes to those sections)
  - Open tasks assigned to me (clickable)
- **"Moji projekti" section:**
  - Last 3-5 projects as cards
  - Each card: project title, donor, status badge, progress bar (approved sections / total sections), due date, quick actions (Open, Duplicate, Archive)

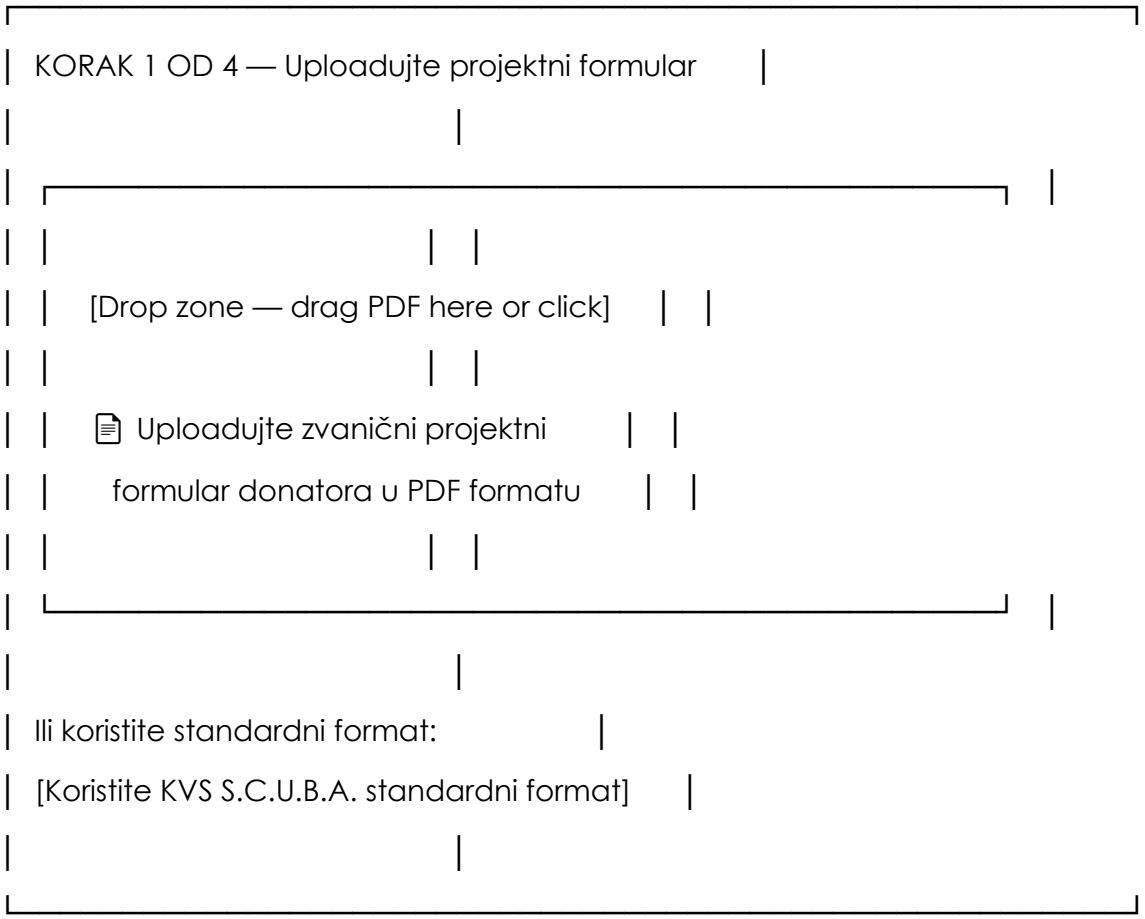
    - "Vidi sve projekte →" link
- **"Zadaci koji čekaju na mene" section:**
    - List of tasks assigned to current user
    - Priority color-coding, due date, project context
    - Mark as done inline
- **Activity feed (right column):**
    - Last 10 activity events across all user's projects
    - "Sekcija odobrena", "Saradnik pridružen", "PDF generisan", etc.
    - Timestamps in relative format ("prije 2 sata")

FEATURE 3: NEW PROJECT WIZARD

Trigger

User clicks "Novi projekat" anywhere in the app → Full-screen multi-step wizard overlay (Framer Motion AnimatePresence)

Step 1: Form Upload



- On upload: send to process-form-upload Edge Function
- Show progress indicator while AI analyzes form
- Display: "Analiziram format formulara... ✓ Identifikovano [N] sekcija"

Step 2: Project Basics

- Project title (auto-suggested from form if detected)
- Donor name
- Donor deadline (date picker)
- Language selection (Bosanski / English)
- Project tags

Step 3: APA Data Collection

- Conversational form UI — questions appear one by one with smooth animation
- Each field has helper text explaining why it's needed
- All 8 mandatory fields + additional fields (see APA Protocol Section 1.3)
- Progress indicator shows % complete
- User can go back and edit any previous answer

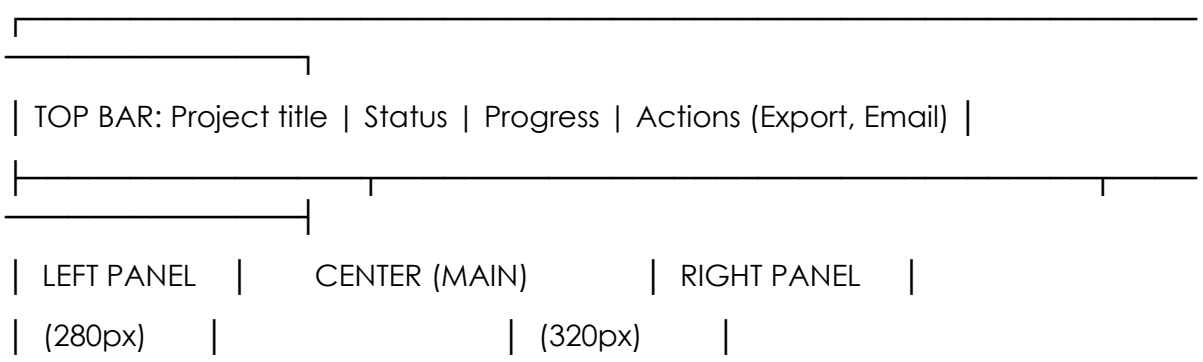
Step 4: Review & Launch

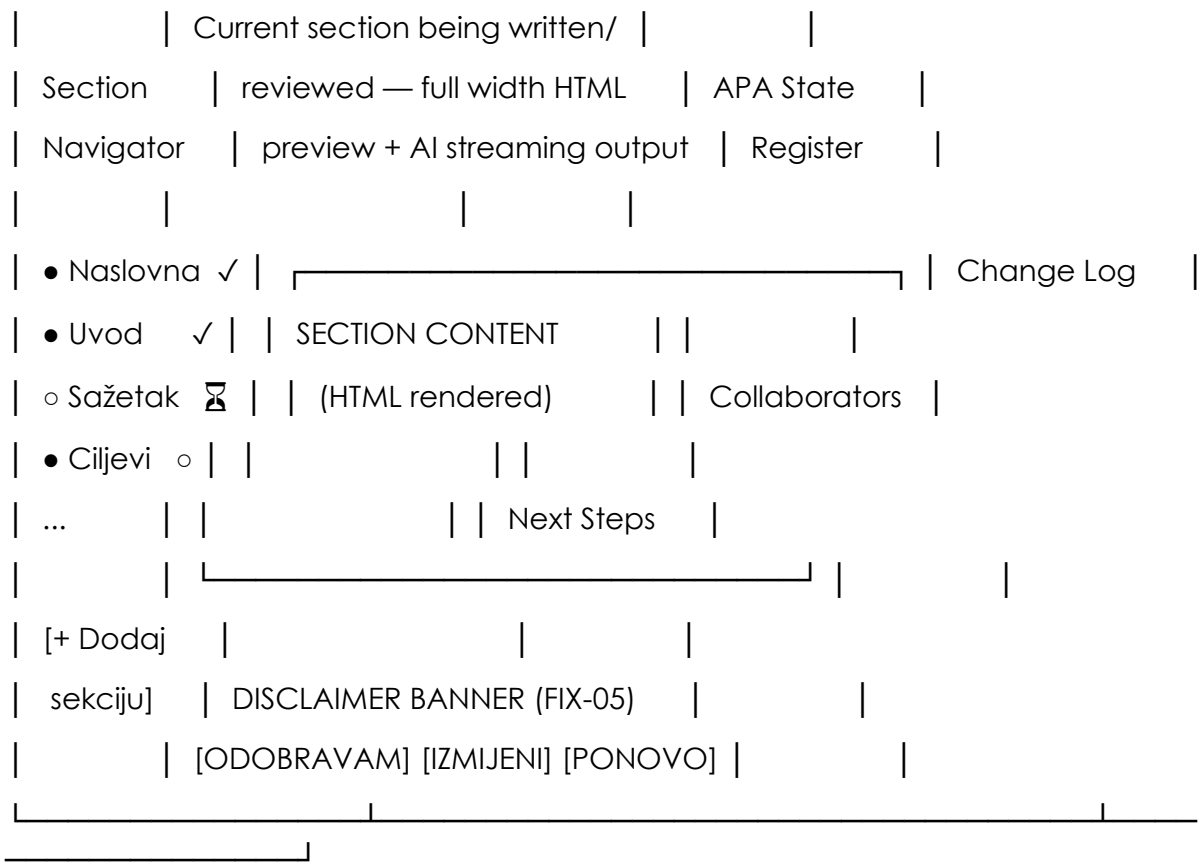
- Summary of all entered data in organized cards
- "Pokrenuti APA/RIP/WA sistem" CTA button
- Estimated time: "Pisanje projekta obično traje 45-90 minuta aktivnog rada"

FEATURE 4: PROJECT EDITOR (/projects/:id/edit)

This is the **core feature** — the most complex and most important screen in the entire app.

Layout (3-column on desktop, stacked on mobile)





Left Panel: Section Navigator

- List of all project sections with status icons:
  - ☐ Pending (not started)
  - 🔄 Generating (AI currently writing)
  - ⏳ Awaiting Approval
  - ☑ Approved
  - ✏ Revision Requested
- Click any section to jump to it
- Drag to reorder optional sections
- "+" button to add optional sections
- Progress bar at top: "8/13 sekcija odobreno"

Center Panel: AI Writing Interface

Phase A — RIP Research (auto-runs after wizard)

- Loading screen: "RIP protokol — Istraživanje podataka za BiH kontekst..."
- Progress indicators per domain: "✓ Zakonodavni okvir | ✓ Ekološki kontekst | ⏳ Demografski podaci..."

- When complete: Expandable "RIP Istraživački sažetak" panel (user can review)

Phase B — Section Writing (WA Protocol) For each section:

1. "Generišem sekciju: [Naziv sekcije]..." — streaming animation
2. AI text streams character by character into the HTML preview
3. When complete: Full section rendered as HTML
4. Disclaimer banner appears (FIX-05)
5. Three action buttons: **"Odobravam"** | **"Izmijeni"** | **"Napiši ponovo"**

Approve Flow:

- Click "Odobravam" → section gets green checkmark → auto-scroll to next section → APA saves to State Register → next section begins generating

Edit Flow (FIX-06):

User clicks "Izmijeni" →

→ Text area appears: "Opišite šta treba promijeniti:"

→ User types change request

→ APA ANALYSIS panel appears:

"📋 APA Analiza izmjene:

Zatražili ste: [user request]

Planiram primijeniti: [APA elaboration]

Pogođene sekcije: [list]

Da li odobravate ovu interpretaciju?"

→ User confirms → WA generates revised section

→ New version shown with change highlighted

Rewrite Flow:

- Click "Napiši ponovo" → Brief reason field (optional) → Section regenerated from scratch

Right Panel: APA State Register

- **State Register tab:** Table showing all sections with current status and version
- **Promjene tab:** Change log table (date, section, change, status, propagated to)
- **Saradnici tab:** List of collaborators and their assigned sections
- **Sljedeći korak tab:** Next required action, who it's waiting on, deadline

Mobile Adaptation

- Single column, sections stacked
- Left panel becomes bottom sheet / drawer
- Right panel becomes collapsible drawer from right

FEATURE 5: PDF GENERATION & EXPORT

This is a **mission-critical feature** — the PDF must be print-ready with zero manual formatting.

PDF Generation Strategy

Use a two-layer approach for maximum fidelity:

Layer 1: HTML/CSS → PDF via Puppeteer (server-side)

- The generate-pdf Edge Function renders the assembled HTML document using a headless browser
- CSS print styles are applied: @media print { ... } with precise page breaks, margins, headers/footers
- Page size: A4 (210mm × 297mm)
- Margins: 25mm top/bottom, 20mm left/right (matching original form)

Layer 2: Form Fidelity The uploaded original PDF form is used as the visual reference. The WA-generated content is rendered inside the exact visual structure of the original form (tables, borders, fonts, colors).

CSS Print Directives

```
@media print {
  @page {
    size: A4;
    margin: 25mm 20mm;
    @top-center {
      content: "KVS „S.C.U.B.A." | " attr(data-project-title);
      font-family: Arial, sans-serif;
      font-size: 9pt;
      color: #666;
    }
    @bottom-right {
      content: "Strana " counter(page) " od " counter(pages);
      font-family: Arial, sans-serif;
```

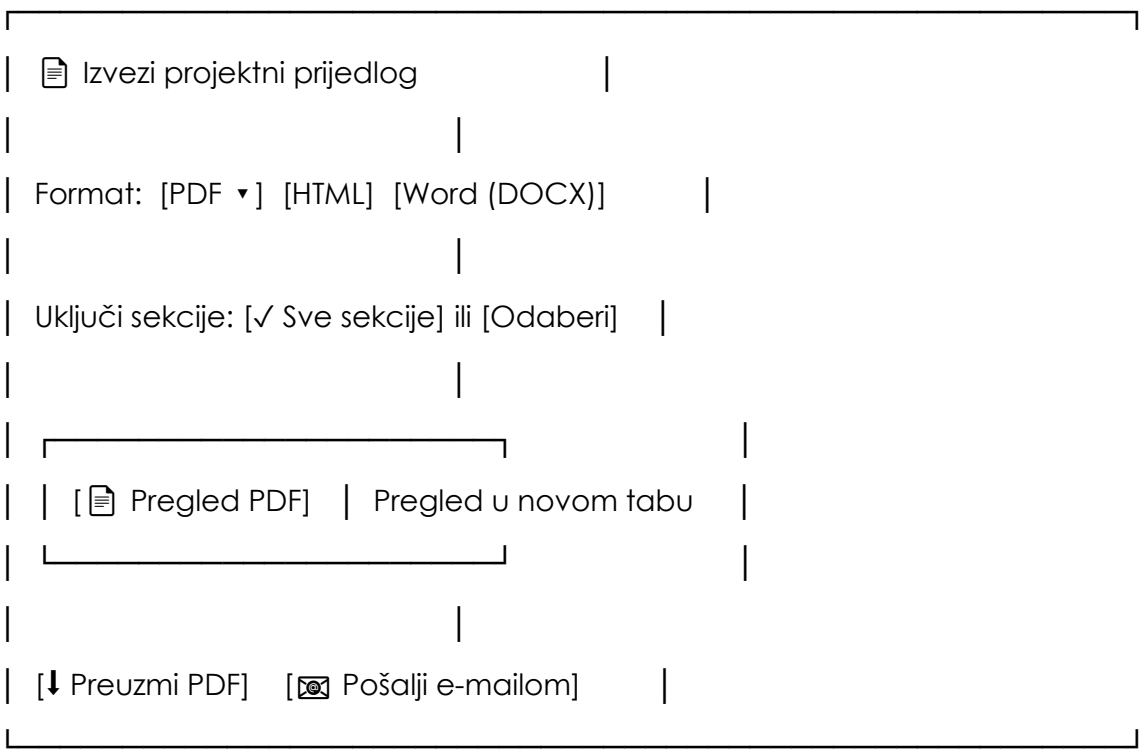
```
    font-size: 9pt;
  }
 }

 .section-container { page-break-inside: avoid; }
 .section-header { page-break-after: avoid; }
 table { page-break-inside: auto; }
 tr { page-break-inside: avoid; }

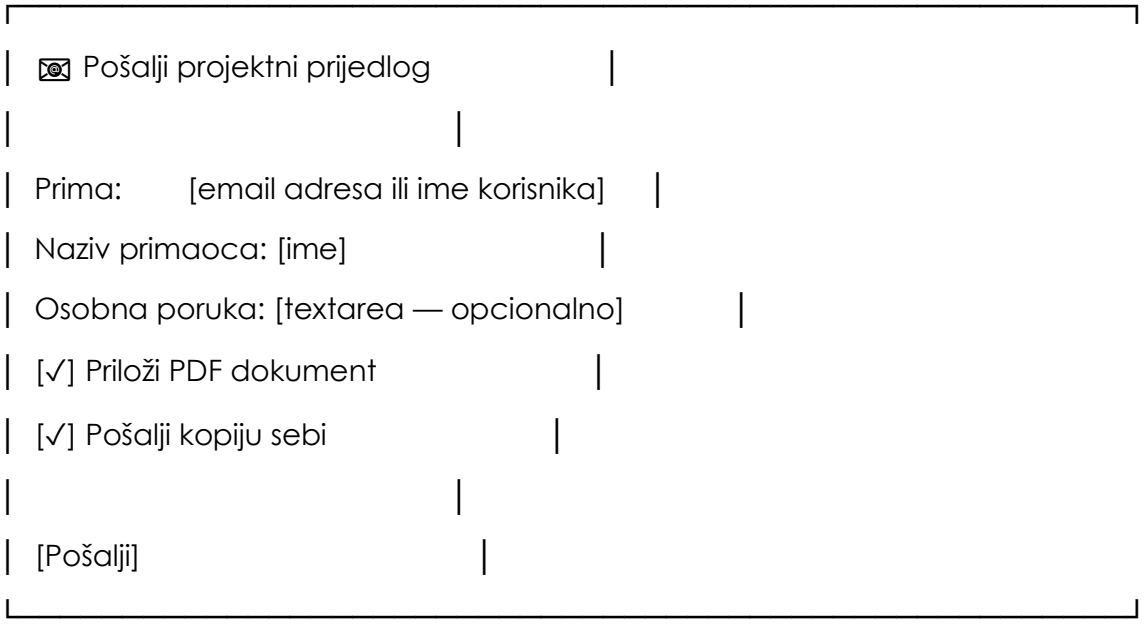
 /* Remove web-specific styling */
 .no-print { display: none !important; }
 body { background: white !important; color: black !important; }

 /* Ensure tables render with borders */
 table, th, td { border: 1px solid #003366 !important; }
 .section-header { background-color: #003366 !important; color: white !important; }
}
```

Export UI



"Pošalji e-mailom" Dialog



Email template: Professional HTML email with KVS S.C.U.B.A. branding, project summary, and PDF attachment. Sent via Resend API.

FEATURE 6: COLLABORATION

Invite Collaborator

- Input: email address
- Assign role: Editor, Reviewer, Viewer
- Assign specific sections (multi-select checkboxes from section list)
- Invitation email sent via Resend

Collaborator Roles

Role	Can View	Can Edit Sections	Can Approve	Can Invite	Can Export
Owner	☑	☑	☑	☑	☑
Editor	☑	☑ (assigned)	✗	✗	☑
Reviewer	☑	✗	☑	✗	☑
Viewer	☑	✗	✗	✗	✗

Real-time Collaboration (Supabase Realtime)

- When a collaborator is viewing the same section: "🟢 Ana Kovač trenutno pregledava ovu sekciju"

- Section status changes instantly reflected for all viewers
- Task completions show in real-time
- Notification bell updates in real-time

Task Board (Kanban)

Columns: **To Do** | **In Progress** | **In Review** | **Done**

Each task card shows:

- Task title
- Assigned user avatar
- Due date (color: green → yellow → red as deadline approaches)
- Section reference
- Priority badge

Users can drag tasks between columns. Status updates persisted to DB.

FEATURE 7: ANALYTICS DASHBOARD (/analytics)

Page Layout

Full-width with responsive grid of charts and data tables.

Charts (all built with Recharts)

1. Projekti po statusu (Pie Chart)

- Donut chart showing: Draft / In Progress / Review / Completed / Archived
- Colors matching status badge colors
- Center: total project count

2. Napredak sekcija (Stacked Bar Chart)

- X-axis: last 5 projects
- Y-axis: section counts
- Stacked bars: Approved / Awaiting / Pending / In Progress

3. Aktivnost tokom vremena (Line Chart)

- X-axis: last 30 days
- Y-axis: sections approved per day
- Smooth curve, gradient fill under line

4. Zadaci po prioritetu (Horizontal Bar Chart)

- Categories: Urgent / High / Normal / Low

- Shows count of open tasks in each priority
- Click to filter task table below

Data Tables

"Čeka se od korisnika" — My Waiting Actions

Projekat	Sekcija	Radnja	Rok	Prioritet
[title]	Budžet	Odobriti sekciju	Prekoračen	Hitno

"Sljedeći korak" — Next Steps per Project

Projekat	Sljedeći korak	Odgovorna osoba	Rok
[title]	Pisanje sekcije "Aktivnosti"	APA sistem	Danas

FEATURE 8: APA HARMONIZATION ENGINE (Institutional Learning)

This is the **intelligence layer** that makes ECO SCUBA better with every completed project.

How It Works

When a user marks a project as **Completed** (all sections approved, PDF downloaded):

1. The project's approved section content is anonymized (personal data stripped)
2. Key patterns extracted: effective argument structures, successful budget formats, proven activity descriptions, strong risk matrices
3. Stored in project_templates table with metadata (donor type, project area, budget range, quality score)
4. Future WA calls receive context: *"Here are N similar successful projects. Use their argumentation style and structure as reference."*

APA Harmonization in Practice

When WA starts writing a new section, APA provides:

HARMONIZATION CONTEXT:

Based on [N] previously approved projects in category [ecology/education],

the following patterns produced the highest-quality approved content:

- [Pattern 1: e.g., "Open with specific statistic about the problem"]

- [Pattern 2: e.g., "Reference BiH legal framework in second paragraph"]

- [Pattern 3: e.g., "Close with measurable outcome statement"]

Reference content from approved projects (anonymized):

[Relevant excerpt 1]

[Relevant excerpt 2]

Quality Scoring

After PDF is exported, prompt user: "Ocijenite kvalitet projekta (1-10)" — stored as quality_score. Used to weight which templates get prioritized in harmonization context.

FEATURE 9: NOTIFICATIONS

Notification Types & Triggers

Type	Trigger	Priority
section_awaiting_approval	WA finishes writing a section	High
collaborator_invited	User invites collaborator	Normal
task_assigned	Task assigned to user	High
task_overdue	Task due date passed	Urgent
project_completed	All sections approved	Normal
change_applied	APA propagates a change	Normal

Notification Bell

- Icon in top nav with badge count
- Click → dropdown panel with recent notifications (max 10)
- "Vidi sve" link → full notifications page
- Mark individual as read / mark all as read
- Real-time updates via Supabase Realtime

In-App Toasts

- Success: Green, bottom-right, 4s duration
- Error: Red, bottom-right, persistent until dismissed
- Info: Blue, bottom-right, 3s duration
- All using a custom toast component (not a library)

FEATURE 10: SETTINGS (/settings)

Profile Settings

- Update name, avatar (upload to Supabase Storage)
- Change password
- Connected accounts (Google OAuth status)
- Language preference (Bosanski / English)

Organization Settings (Admin only)

- Organization name
- Default project language
- Email signature for PDF exports

Notification Preferences

- Toggle each notification type on/off
- Email notifications: on/off

Security

- Active sessions list with ability to revoke
- Two-factor authentication (TOTP) — implement if Supabase supports in project tier

Danger Zone

- Delete all projects (requires confirmation: type "OBRISI SVE")
- Delete account (requires confirmation)

AI INTEGRATION — CLAUDE API

System Prompt Architecture

The full APA + RIP + WA System Instructions (v2.0) must be embedded as the system prompt for all Claude API calls. The system prompt is stored in src/lib/apa-system-instructions.ts and injected into every API call.

Edge Function: ai-generate-section

```
import Anthropic from 'npm:@anthropic-ai/sdk';


Deno.serve(async (req) => {
  // Validate auth
  const authHeader = req.headers.get('Authorization');
  // ... validate with Supabase
```

```
  const { project_id, section_key, protocol, conversation_history, project_context } =
await req.json();


  const client = new Anthropic({ apiKey: Deno.env.get('ANTHROPIC_API_KEY') });


  // Build messages array
  const messages = [
    ...conversation_history,
    {
      role: 'user',
      content: buildPromptForSection(section_key, protocol, project_context)
    }
  ];


  // Stream response
  const stream = await client.messages.create({
    model: 'claude-sonnet-4-6-20251001',
    max_tokens: 4096,
    system: APA_SYSTEM_INSTRUCTIONS, // Full v2.0 system prompt
    messages,
    stream: true
  });


  // Return SSE stream
  const encoder = new TextEncoder();
  const readable = new ReadableStream({
    async start(controller) {
      for await (const event of stream) {
        if (event.type === 'content_block_delta') {
          controller.enqueue(encoder.encode(
```

```
          `data: ${JSON.stringify({ delta: event.delta.text })}\n\n`
        ));
      }
      if (event.type === 'message_stop') {
        controller.enqueue(encoder.encode('data: [DONE]\n\n'));
        controller.close();
      }
    }
  }
 });

 return new Response(readable, {
  headers: {
   'Content-Type': 'text/event-stream',
   'Cache-Control': 'no-cache',
   'Connection': 'keep-alive',
   'Access-Control-Allow-Origin': '*',
  }
 });
});
```

Frontend Streaming Hook

```
// src/hooks/useAIStream.ts
export function useAIStream() {
 const [content, setContent] = useState('');
 const [isStreaming, setIsStreaming] = useState(false);

 const streamSection = async (params: StreamParams) => {
  setIsStreaming(true);
  setContent('');
```

```
    const response = await fetch(`${SUPABASE_URL}/functions/v1/ai-generate-section`, {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${session.access_token}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(params),
    });

    const reader = response.body?.getReader();
    const decoder = new TextDecoder();

    while (true) {
      const { done, value } = await reader!.read();
      if (done) break;

      const chunk = decoder.decode(value);
      const lines = chunk.split('\n');

      for (const line of lines) {
        if (line.startsWith('data: ')) {
          const data = line.slice(6);
          if (data === '[DONE]') { setIsStreaming(false); return; }
          const parsed = JSON.parse(data);
          if (parsed.delta) setContent(prev => prev + parsed.delta);
        }
      }
    }
  };
```

```
  return { content, isStreaming, streamSection };
}
```

SECURITY REQUIREMENTS

Authentication Security

- [ ] All API calls require valid Supabase JWT
- [ ] JWT validated in every Edge Function
- [ ] Session expiry: 1 hour (access token), 7 days (refresh token)
- [ ] Google OAuth state parameter validated (CSRF protection)
- [ ] Password requirements: min 8 chars, at least 1 uppercase, 1 number

Database Security

- [ ] Row Level Security enabled on ALL tables
- [ ] Users can only access their own data and data they're collaborating on
- [ ] No direct database access from frontend — all via Supabase client
- [ ] Sensitive data (AI conversations) never exposed in URL params
- [ ] Database backups enabled in Supabase

API Security

- [ ] Anthropic API key stored ONLY in Supabase Edge Function secrets — never in frontend
- [ ] Resend API key stored ONLY in Supabase Edge Function secrets
- [ ] CORS configured to allow only app domain
- [ ] Rate limiting: max 10 AI generation requests per user per hour
- [ ] Input validation with Zod on all API calls

File Upload Security

- [ ] File type validation: only PDF files accepted for form upload
- [ ] File size limit: 10MB max
- [ ] Virus scanning: implement via Supabase Storage policies
- [ ] Uploaded files stored in private Supabase Storage bucket
- [ ] Signed URLs with 1-hour expiry for file access

Frontend Security

- [ ] Content Security Policy (CSP) headers configured

- [ ] XSS protection: all user-generated content sanitized with DOMPurify before rendering
- [ ] HTTPS enforced (Lovable handles this)
- [ ] No sensitive data in localStorage (only Supabase session)
- [ ] Error messages never expose internal implementation details

PDF Security

- [ ] Generated PDFs stored in private Supabase Storage bucket
- [ ] PDF download requires valid auth + project access check
- [ ] PDF URLs are signed with expiry — never permanent public URLs

---

RESPONSIVE DESIGN BREAKPOINTS

/* Mobile first */

/* xs: 0-639px — single column, full screen */

/* sm: 640-767px — same as mobile with slightly more space */

/* md: 768-1023px — two columns, sidebar becomes overlay drawer */

/* lg: 1024-1279px — three columns, full layout */

/* xl: 1280px+ — wide layout with more breathing room */

Mobile Adaptations

- Sidebar: becomes bottom tab bar on mobile
- Project editor: single column, section nav becomes floating FAB
- Charts: horizontal scroll on small screens
- Tables: card layout on mobile (each row becomes a card)

PERFORMANCE REQUIREMENTS

- [ ] Initial page load < 3 seconds (LCP)
- [ ] First Contentful Paint < 1.5 seconds
- [ ] Lazy load all route components (React.lazy + Suspense)
- [ ] Skeleton loaders for all async data (never blank screens)
- [ ] AI streaming must show first token within 2 seconds
- [ ] Images optimized and lazy loaded
- [ ] Bundle size < 500KB gzipped (split code aggressively)

- [ ] Supabase queries indexed (all foreign keys, status fields)

ACCESSIBILITY

- [ ] WCAG 2.1 AA compliance
- [ ] All interactive elements keyboard navigable (Tab + Enter + Space + Escape)
- [ ] Focus visible indicator on all focusable elements
- [ ] aria-labels on all icon buttons
- [ ] Color is never the only means of conveying information
- [ ] Screen reader compatible (semantic HTML, ARIA roles)
- [ ] Skip to main content link

ERROR HANDLING

AI Generation Errors

- Network timeout: "Generisanje je prekinuto. Da li želite pokušati ponovo?"
- Rate limit exceeded: "Dostigli ste limit zahtjeva. Pokušajte ponovo za [X] minuta."
- API error: "Došlo je do greške pri komunikaciji s AI sistemom. [Pokušaj ponovo]"
- Always preserve partial content — never lose work

Form Upload Errors

- Wrong file type: "Molimo uploadujte PDF dokument."
- File too large: "Dokument je prevelik. Maksimalna veličina je 10MB."
- Corrupted PDF: "PDF dokument se ne može pročitati. Pokušajte s drugim dokumentom."

General Error Boundary

Wrap entire app in React Error Boundary. On uncaught error: friendly error page with "Prijavite grešku" button.

ENVIRONMENT VARIABLES

```
# Supabase

VITE_SUPABASE_URL=https://[project-id].supabase.co

VITE_SUPABASE_ANON_KEY=[anon-key]
```

In Supabase Edge Function secrets (NOT in .env):

ANTHROPIC_API_KEY=[key]

RESEND_API_KEY=[key]

INITIAL DATA SEED

On first app launch (or via admin panel), seed:

-- Default section definitions for all projects

INSERT INTO public.project_sections_template (section_key, section_title_bs, display_order, is_optional) VALUES

('naslovna_strana', 'Naslovna strana', 0, false),

('uvod', 'Uvod', 1, false),

('sazetak', 'Sažetak', 2, false),

('potreba_problem', 'Potreba/problem u lokalnoj zajednici', 3, false),

('razlozi_znacaj', 'Razlozi i značaj projekta za lokalnu zajednicu', 4, false),

('ciljevi_projekta', 'Ciljevi projekta', 5, false),

('ciljna_grupa', 'Ciljna grupa', 6, false),

('sveukupni_cilj', 'Sveukupni cilj projekta', 7, false),

('specificni_ciljevi', 'Specifični ciljevi projekta', 8, false),

('ocekivani_rezultati', 'Očekivani rezultati', 9, false),

('aktivnosti', 'Aktivnosti', 10, false),

('pretpostavke_rizici', 'Pretpostavke i rizici', 11, false),

('trajanje_projekta', 'Trajanje projekta', 12, false),

('pracenje_izvjestavanje', 'Praćenje provedbe i izvještavanje', 13, false),

('budzet', 'Budžet', 14, false),

('vidljivost_promocija', 'Vidljivost (Promocija projekta)', 15, false),

('lista_aneksa', 'Lista aneksa', 16, false),

-- Optional sections

('metodologija', 'Metodologija', 17, true),

('odrzivost_projekta', 'Održivost projekta', 18, true),

('rodna_ravnopravnost', 'Rodna ravnopravnost i socijalna inkluzija', 19, true),

('ekoloski_uticaj', 'Ekološki uticaj projekta', 20, true),

('saradnja_institucijama', 'Saradnja s institucijama', 21, true),

('informacije_nositelju', 'Informacije o nositelju projekta', 22, true);

BUILD & DEPLOYMENT

Lovable.dev Configuration

- Build command: npm run build
- Output directory: dist
- Node version: 18+

Environment Setup in Lovable

1. Add Supabase integration via Lovable's Supabase connector
2. Set VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY in project settings
3. Add ANTHROPIC_API_KEY and RESEND_API_KEY to Supabase Edge Function secrets

Supabase Setup Checklist

- [ ] Run all SQL from database schema section
- [ ] Enable Google OAuth provider in Supabase Auth settings
- [ ] Create Storage buckets: form-uploads (private), generated-pdfs (private), avatars (public)
- [ ] Deploy all 4 Edge Functions
- [ ] Set secrets for Edge Functions

FINAL CHECKLIST — BUILD COMPLETENESS

Before considering the app complete, verify:

Authentication

- [ ] Email/password registration with email verification works
- [ ] Google OAuth login works
- [ ] Protected routes redirect unauthenticated users
- [ ] Session persists across browser refresh

Core APA+RIP+WA Flow

- [ ] PDF form upload works and triggers AI analysis
- [ ] APA data collection wizard collects all required fields

- [ ] RIP research phase runs and displays progress
- [ ] WA writes sections with streaming animation
- [ ] Disclaimer banner (FIX-05) appears after every section
- [ ] Approve/Edit/Rewrite flow works correctly
- [ ] Edit flow triggers APA analysis → elaboration → user confirmation → propagation (FIX-06)
- [ ] APA State Register persists all approved sections (FIX-07)
- [ ] Final assembly reflects all changes (FIX-08)
- [ ] All section names appear in Bosnian (FIX-02)

PDF Export

- [ ] PDF generates with correct formatting
- [ ] PDF is print-ready (test by opening in browser and using print preview)
- [ ] All sections included in correct order
- [ ] KVS S.C.U.B.A. header/footer on all pages
- [ ] Page numbers correct
- [ ] Download works

Email

- [ ] Email sends successfully with PDF attachment
- [ ] Email renders correctly in Gmail/Outlook
- [ ] Professional HTML email template displays KVS S.C.U.B.A. branding

Collaboration

- [ ] Invite collaborator by email works
- [ ] Collaborator receives invitation email
- [ ] Role-based permissions enforced
- [ ] Real-time section status updates work
- [ ] Task board CRUD operations work

Analytics

- [ ] All charts render with real data
- [ ] "Waiting for me" table shows correct tasks
- [ ] "Next steps" table is accurate per project

Security

- [ ] RLS prevents cross-user data access (test with two different accounts)
- [ ] API keys not visible in browser network requests
- [ ] File upload validates file type and size

Performance

- [ ] No visible lag when switching between sections
- [ ] AI streaming starts within 2 seconds
- [ ] Charts load without blocking main content

End of ECO SCUBA — ACA Build Instructions v1.0 For: Klub vodenih sportova „S.C.U.B.A.", Sarajevo, BiH Build in: Lovable.dev | AI: Anthropic Claude | Backend: Supabase